

Sprint 5 — Execução

Arquitetura Híbrida de Baixo Custo para Triagem Automatizada e Classificação de Chamados Acadêmicos

Aluno: Caio Vitor Pinheiro Moraes — Matrícula: 04179424
Parte Acadêmica (evidências do experimento) e Parte Profissional (MVP Alpha)

Parte Acadêmica — Execução da metodologia

1. O que foi executado

Seguindo a metodologia definida nas Sprints 2 a 4, a arquitetura híbrida foi implementada e submetida ao experimento planejado. Foram executadas as etapas: (i) construção e rotulagem do dataset; (ii) implementação dos três classificadores comparados (regras puras, PLN isolado e híbrido); (iii) validação cruzada com métricas de recuperação de informação; (iv) teste de robustez a erros de digitação; (v) medição de latência em processo e sob carga HTTP.

- Dataset: 135 chamados acadêmicos sintéticos (45 por setor: Financeiro, Secretaria, Coordenação), escritos à mão com variação de linguagem, rotulados com setor correto e urgência (alta/média/baixa) — arquivo data/dataset.csv.
- Regras: léxico ponderado por setor (termos fortes e fracos) sobre texto normalizado; decide sozinho quando um setor vence por margem ≥ 2 pontos.
- PLN leve: TF-IDF (palavras 1-2 gramas + n-gramas de caracteres 2-5) com Regressão Logística; acionado quando as regras são incertas. Chamados com confiança $< 0,5$ são sinalizados para revisão manual.
- Protocolo: validação cruzada estratificada 5-fold repetida 5 vezes (25 avaliações por método). O classificador de PLN é treinado apenas nas dobras de treino. Cenário B aplica ruído de digitação (troca/omissão/duplicação de letras em $\sim 30\%$ das palavras) apenas aos textos de teste.

2. Resultados

2.1 Acurácia no direcionamento de setor

Método	Texto limpo (média $\pm$ dp)	Com erros de digitação (média $\pm$ dp)
Regras puras	92.6% $\pm$ 4.1	76.7% $\pm$ 7.2
PLN isolado	80.6% $\pm$ 6.8	76.7% $\pm$ 7.6
Híbrido (proposto)	92.0% $\pm$ 4.8	86.1% $\pm$ 6.1

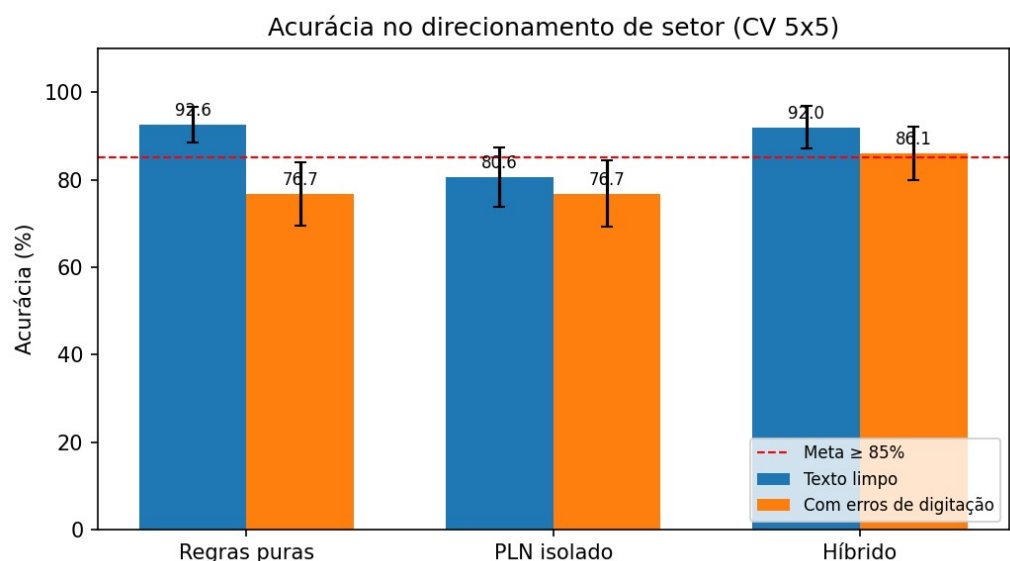


Figura 1 — Acurácia por método e cenário (validação cruzada 5x5). Linha tracejada: meta de 85%.

2.2 Precisão, revocação e F1 por setor (algoritmo híbrido)

Setor	P (limpo)	R (limpo)	F1 (limpo)	P (ruído)	R (ruído)	F1 (ruído)
Financeiro	0.952	0.880	0.915	0.913	0.836	0.872
Secretaria	0.898	0.902	0.900	0.810	0.831	0.820
Coordenação	0.913	0.978	0.944	0.866	0.916	0.890

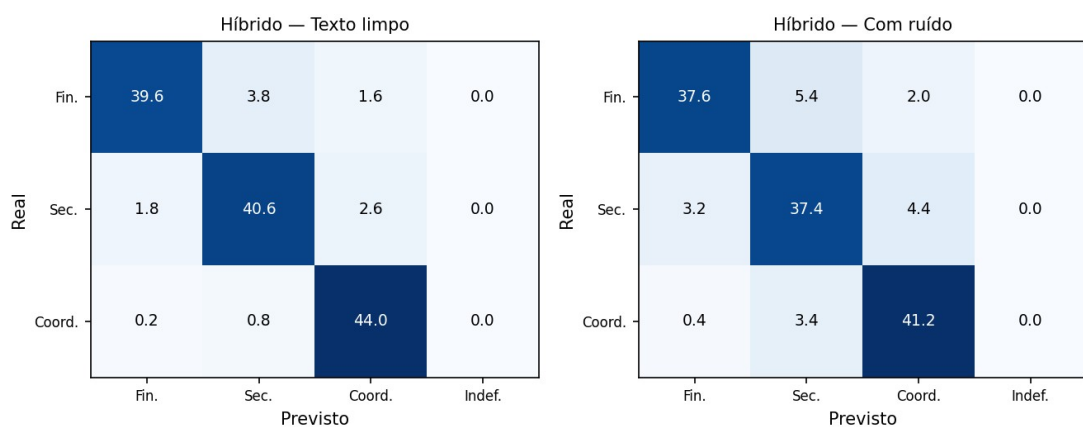


Figura 2 — Matriz de confusão do algoritmo híbrido (média por repetição; 45 chamados por setor por repetição).

Cobertura das regras no híbrido: 81.5% dos chamados foram resolvidos só pelas regras em texto limpo e 67.1% com ruído (o restante acionou o PLN). Acurácia da classificação de urgência (regras): 74.1% ± 10.0.

2.3 Desempenho (latência)

Método	Latência média (ms)	p95 (ms)	Amostras
Regras puras	0.044	0.059	1350
PLN isolado	1.521	2.178	1350

Híbrido (proposto)	0.339	1.661	1350
--------------------	-------	-------	------

Requisições simultâneas	Total de requisições	Sucesso	p50 (ms)	p95 (ms)	Vazão (req/s)
1	100	100/100	3.0	5.0	279.0
10	200	200/200	9.3	118.8	304.4
50	300	300/300	42.8	570.8	295.5

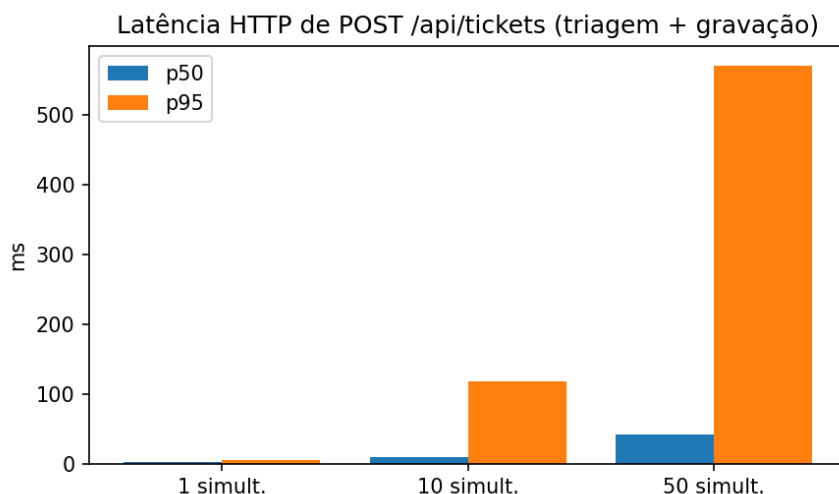


Figura 3 — Latência de ponta a ponta de POST /api/tickets (triagem + gravação no banco) sob concorrência.

Medição feita em máquina única, com o servidor de desenvolvimento do Flask e SQLite; os valores servem como ordem de grandeza e devem ser repetidos no ambiente de implantação.

3. Análise dos resultados

- A meta de acurácia $\geq 85\%$ foi atingida pelo algoritmo híbrido nos dois cenários (92.0% em texto limpo e 86.1% com ruído), embora, no cenário com ruído, a margem seja estreita em relação ao desvio-padrão entre dobras.
- Em texto limpo o híbrido não superou as regras puras (92,0% vs 92,6%, diferença dentro do desvio-padrão). A vantagem do híbrido aparece com erros de digitação: 86,1% contra 76,7% tanto das regras quanto do PLN isolado (+9,4 p.p.), o que sustenta a hipótese central: regras dão previsibilidade e custo quase nulo; o PLN cobre os casos em que as regras falham.
- O PLN isolado teve o pior desempenho em texto limpo (80,6%), coerente com a literatura: exige base histórica maior do que as ~108 amostras de treino por dobra usadas aqui.
- O custo computacional é baixo: triagem média de 0,34 ms no híbrido (0,04 ms só regras; 1,5 ms PLN), e 100% de sucesso nas requisições HTTP com até 50 simultâneas, com aumento de latência (p95) conforme a concorrência cresce.
- A classificação de urgência por regras (74,1%) é o ponto mais fraco e é candidata a melhoria (classificador próprio).

4. Ameaças à validade e limitações (declaradas)

- Dataset sintético e pequeno (135 chamados): os resultados indicam viabilidade, não desempenho em produção. O léxico de regras e o dataset foram elaborados por uma mesma origem de conhecimento, o que tende a favorecer as regras (otimismo). Para a versão final do TCC recomenda-se acrescentar chamados reais anonimizados e/ou textos escritos por terceiros e repetir o experimento.
- O ruído de digitação é simulado por um gerador simples; ele não cobre abreviações, gírias ou textos longos.
- O baseline manual (comparação com triagem humana) ainda NÃO foi medido: o protocolo e o script estão prontos (experimentos/baseline_manual.py), mas exigem voluntários reais. Sem esses dados, a afirmação de “redução drástica de tempo em relação ao manual” não pode ser feita com evidência — apenas o tempo do sistema (milissegundos) foi medido.
- Latência medida em ambiente de desenvolvimento, sem rede real entre cliente e servidor.

5. Reprodutibilidade

Todos os números acima foram gerados por `python -m experimentos.experimento` (sementes fixas) e estão em `evidencias/resultados_experimento.json`; o log completo está em `evidencias/log_experimento.txt`.

Parte Profissional — MVP Alpha

1. O que funciona

O MVP Alpha é a primeira versão executável do sistema: o fluxo central (aluno abre chamado → API tria → chamado entra na fila do setor → atendente responde) funciona de ponta a ponta, com interface web.

Requisito do MVP (Sprint 3)	Situação no Alpha	Evidência
Autenticação e perfis (Aluno / Atendente) com JWT	Funcionando	Teste 01-03, 06; tela de login
Gestão de tickets (criar e acompanhar)	Funcionando	Teste 04; Figura 4
API de triagem automática (setor + urgência)	Funcionando	Teste 04, 07; Figuras 1-3
Painel do atendente (fila por setor, responder, status)	Funcionando	Teste 04-05; Figuras 5-6
Reencaminhamento manual de setor (correção da triagem)	Funcionando (extra)	Teste 05
Notificação por e-mail / busca por palavra-chave / métricas avançadas	Não iniciado (desejáveis)	—

2. Arquitetura e stack

- Backend: Python 3.12 + Flask (API REST). Autenticação: JWT (PyJWT, HS256, expiração de 8 h) e senhas com hash (Werkzeug).
- Banco de dados: SQLite (3 tabelas: users, tickets, respostas). Containerização: Dockerfile com gunicorn (ainda não executado neste ambiente).
- Frontend: HTML5 + CSS3 + JavaScript puro em página única, servida pela própria API.
- Motor de triagem: módulo app/triagem.py (regras + scikit-learn), treinado no início da aplicação a partir de data/dataset.csv.
- Desvio em relação ao planejado: as Sprints 2 e 3 previam Java/Spring Boot + PostgreSQL. Para garantir um protótipo executável e testado nesta etapa, o Alpha foi feito em Python/Flask + SQLite. A arquitetura (API REST, JWT, banco relacional, contêiner, triagem híbrida) é equivalente e a migração para Spring Boot/PostgreSQL permanece no plano.

3. Evidências de funcionamento

Testes automatizados: 8 testes de integração (evidencias/testes_api.txt) passaram, cobrindo login inválido, rotas protegidas, cadastro e e-mail duplicado, fluxo completo de triagem e painel, isolamento entre setores e entre alunos, reencaminhamento, validações e tolerância a erros de digitação.

Chamados Acadêmicos — Triagem Híbrida

Aluno Demo (aluno)Sair

Abrir novo chamado

Assunto

Ex.: Segunda via de boleto

Descreva seu problema

Explique sua solicitação com detalhes...

Enviar chamado

☒

 Chamado #3 direcionado automaticamente para **Coordenação** (urgência baixa) em 0.166 ms — método: regras.

Meus chamados

#1 • Boleto vencido

Urgência alta

Financeiro

Aberto

30/09/2026, 20:22:52

#2 • Histórico escolar

Urgência média

Secretaria

Aberto

30/09/2026, 20:22:53

#3 • Revisão de nota

Urgência baixa

Coordenação

Aberto

30/09/2026, 20:22:53

Figura 4 — Aluno abre chamados; o sistema informa setor, urgência, método e tempo de triagem.

Chamados Acadêmicos — Triagem Híbrida

Supervisor Geral (atendente · Todos)Sair

Painel — todos os setores

Total

3

Abertos

3

Em andamento

0

Resolvidos

0

Urgência alta

1

Triagem média (ms)

2.933

Reencaminhados

0

Fila de chamados

Todos os status

Todos os setores

#1 • Boleto vencido

Urgência alta

Financeiro

Aberto

Aluno Demo · 30/09/2026, 20:22:52

#2 • Histórico escolar

Urgência média

Secretaria

Aberto

Aluno Demo · 30/09/2026, 20:22:53

#3 • Revisão de nota

Urgência baixa

Coordenação

Aberto

Aluno Demo · 30/09/2026, 20:22:53

Figura 5 — Painel do atendente com indicadores e fila ordenada por urgência.

Chamados Acadêmicos — Triagem Híbrida

Supervisor Geral (atendente · Todos) Sair

← Voltar

#1 - Boleto vencido

Urgência alta Financeiro Em andamento

Aberto por Aluno Demo em 30/09/2026, 20:22:52 · triagem: regras, 0.309 ms

O boleto da mensalidade venceu ontem e o sistema não deixa emitir outro, preciso pagar hoje

Em andamento Atualizar status Financeiro Reencaminhar setor

Conversa

Supervisor Geral 30/09/2026, 20:22:55
Olá! Enviamos a segunda via do boleto para o seu e-mail.

Nova mensagem

Enviar

Figura 6 — Detalhe do chamado: resposta do atendente e mudança automática de status para “em andamento”.

4. Como executar (demonstração)

- `pip install -r requirements.txt → python -m app.app → abrir http://localhost:5000`
- Aluno: `aluno@unama.test / aluno123`. Atendentes: `financeiro@`, `secretaria@`, `coordenacao@` ou `supervisor@unama.test / atende123`.

5. Próximos passos

- Executar o baseline manual (30 chamados, 2-3 avaliadores) e incluir a comparação no relatório final.
- Ampliar e diversificar o dataset (chamados reais anonimizados) e repetir o experimento.
- Melhorar a classificação de urgência; implementar notificações por e-mail e busca por palavra-chave.
- Migrar para Spring Boot + PostgreSQL e publicar (Docker/Render) conforme o plano de portfólio.